

COMP5349– Cloud Computing

Week 12: Kubernetes

Dr. Ying Zhou

The University of Sydney

Table of Contents

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the **University of Sydney** pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

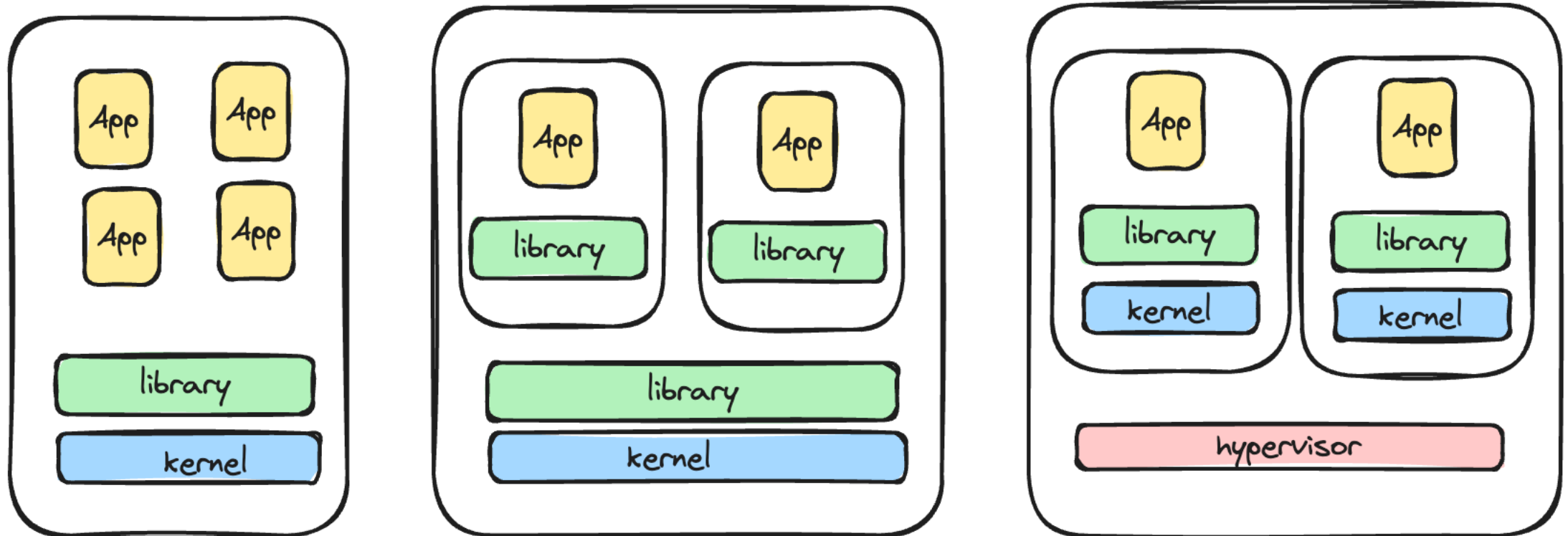
- 01 Kubernetes Overview
- 02 Pods
- 03 Service and Deployment
- 04 Kubernetes Manifest file

Kubernetes Overview

Kubernetes started in Google

- Kubernetes is a Greek word for *pilot* or *helmsman*
- Google has been running applications in containers since early 2000
- Back in 2014, it was reported that they start two billion containers every week.
- The predecessor of **Kubernetes** is **Borg** (and **Omega**) , Google's internal container-oriented cluster management system.
 - “admits, schedules, starts, restarts, and monitors the *full range of applications* that Google runs.”
 - They also help them achieving better utilization of the infrastructure
- Kubernetes was introduced in 2014

Review of application running options



Abstracting the infrastructure away

- Kubernetes provides an abstraction layer over the underlying hardware to both users and applications.

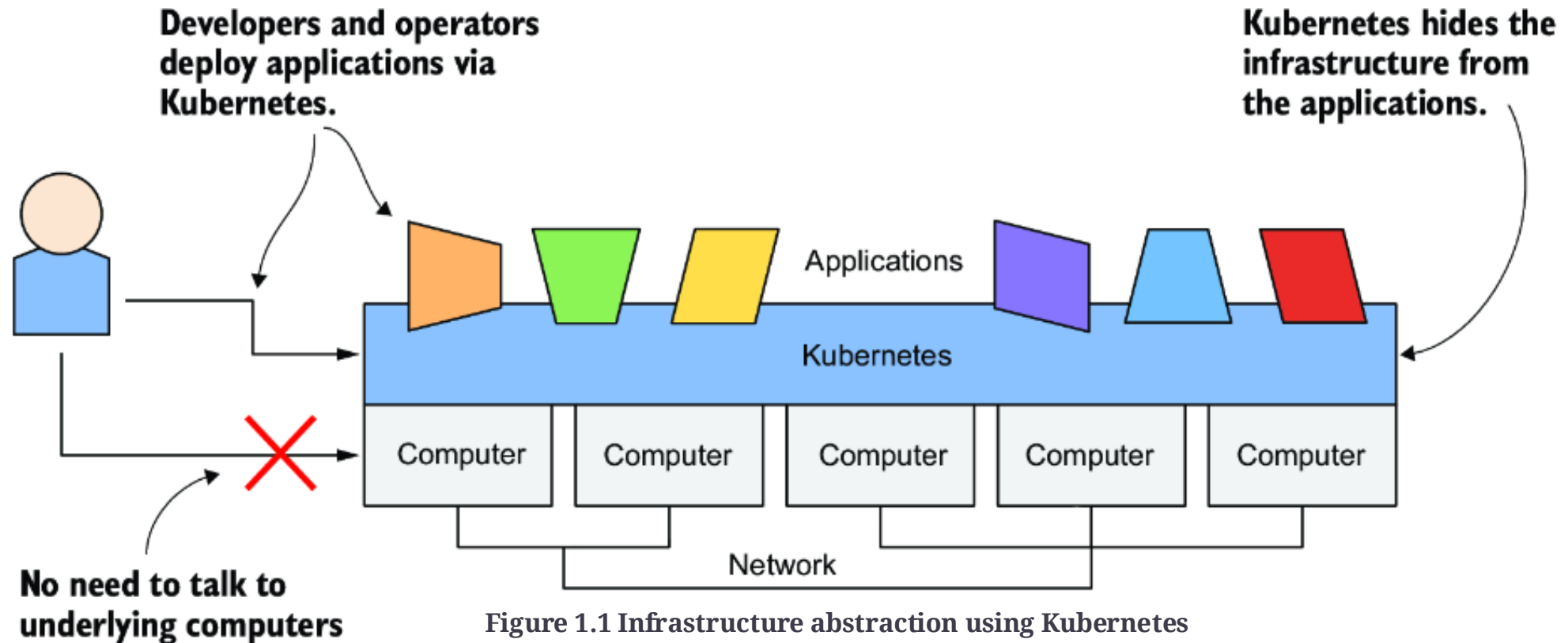


Figure 1.1 Infrastructure abstraction using Kubernetes

Deploying applications declaratively

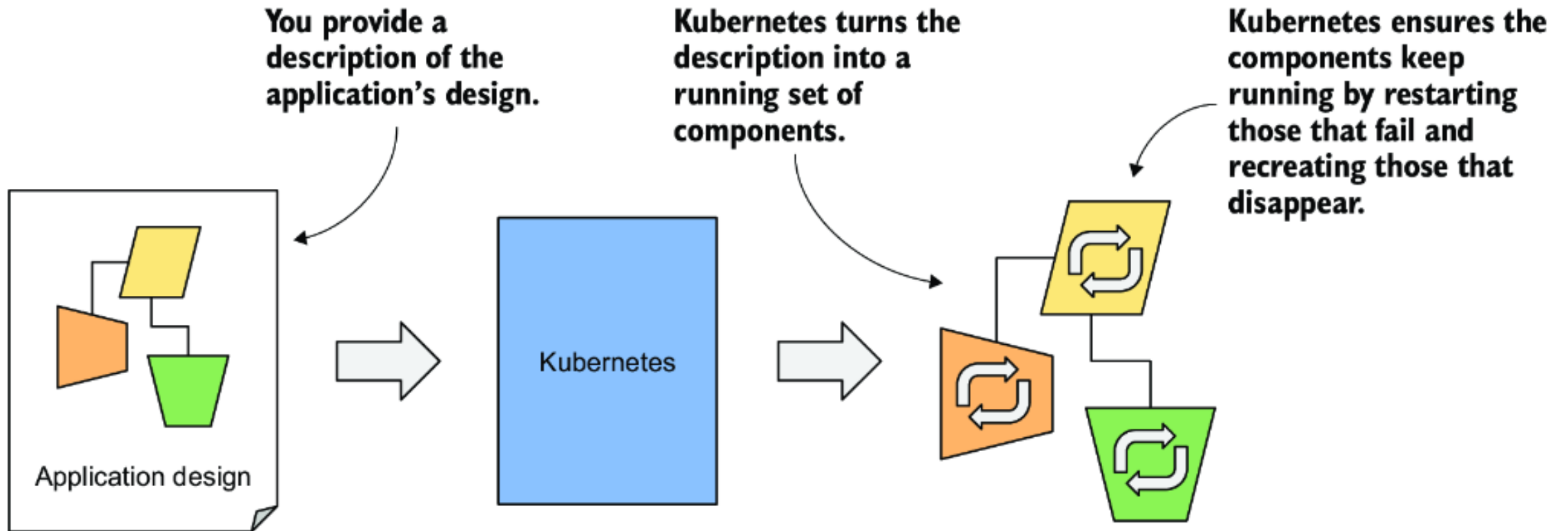


Figure 1.2 The declarative model of application deployment

Changes are managed automatically

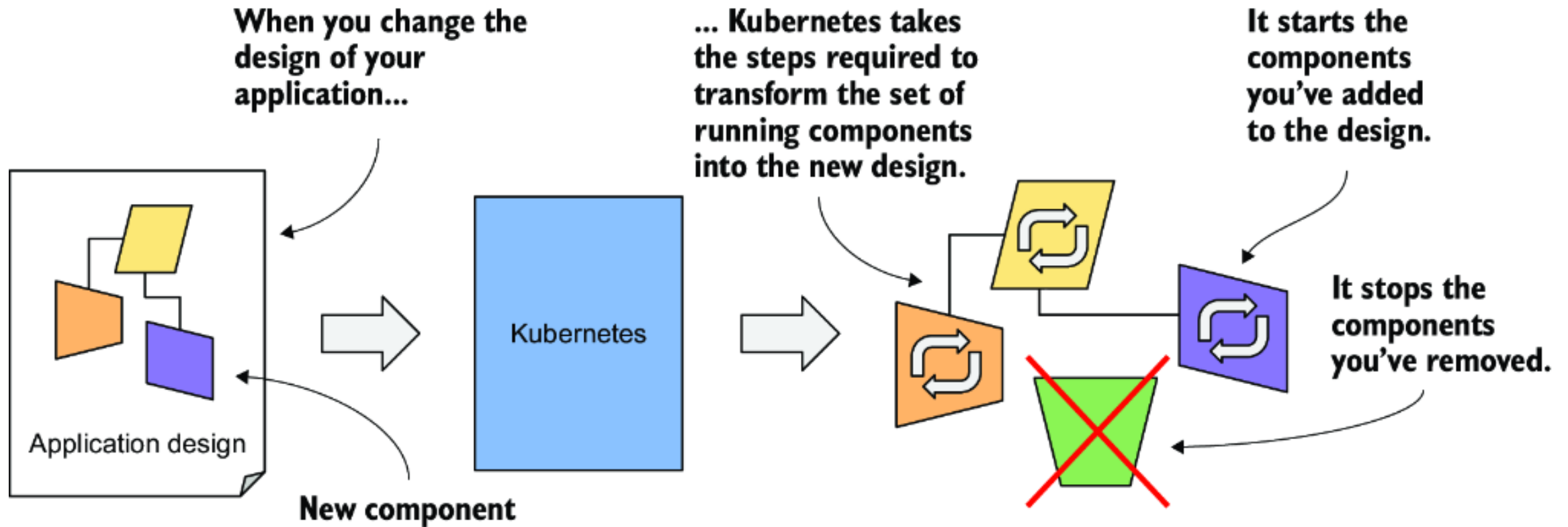
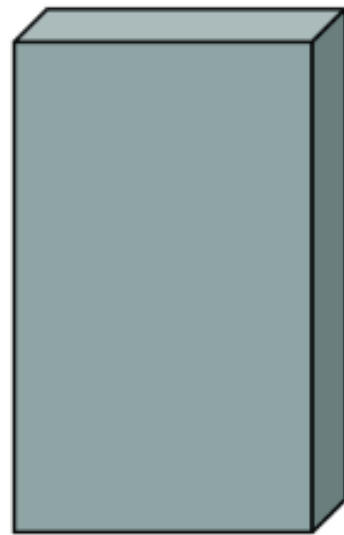


Figure 1.3 Changes in the description are reflected in the running application.

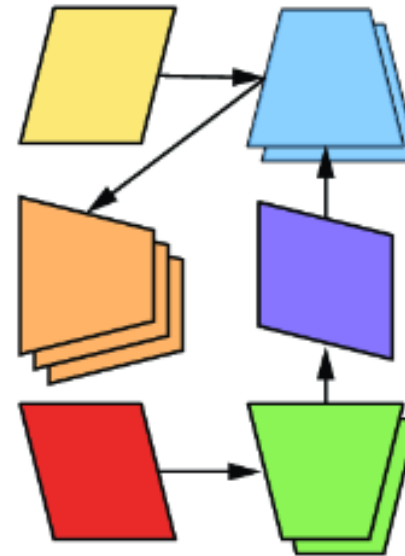
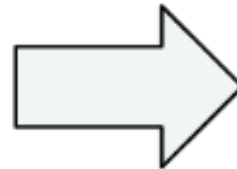
Automating the management of microservices

A monolithic application consists of one or a very small number of applications.

Monolithic applications can be managed manually.



Monolithic application



Microservices

Each microservice is an **application** in its own right. It must be installed, configured, and managed separately.

Microservices require automated management due to their multiplicity and distributed nature.

Figure 1.6 Comparing monolithic applications with microservices

Standardizing the cloud

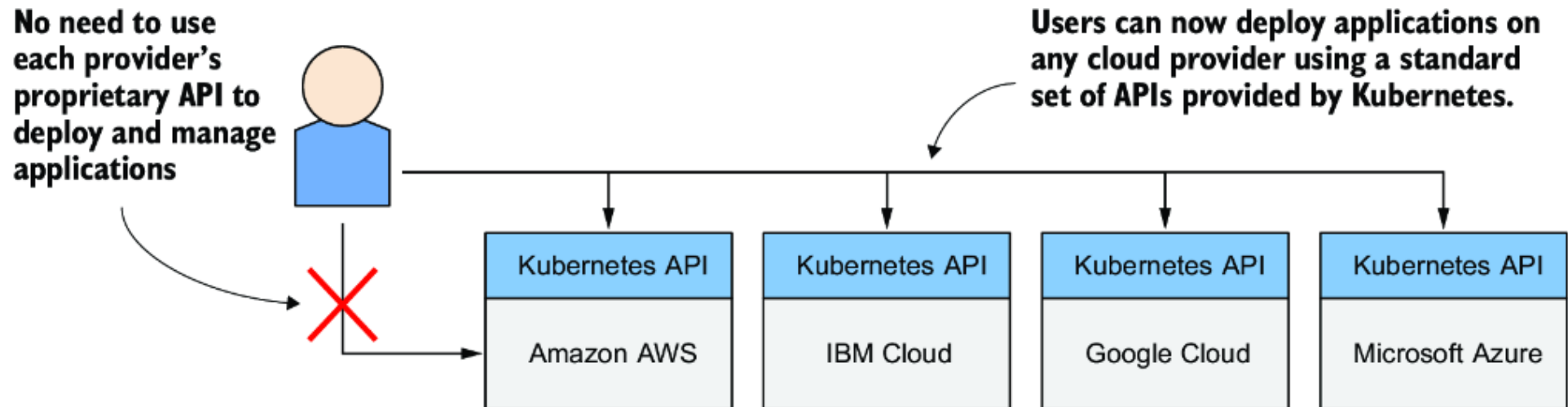


Figure 1.7 Kubernetes has standardized how you deploy applications on cloud provide

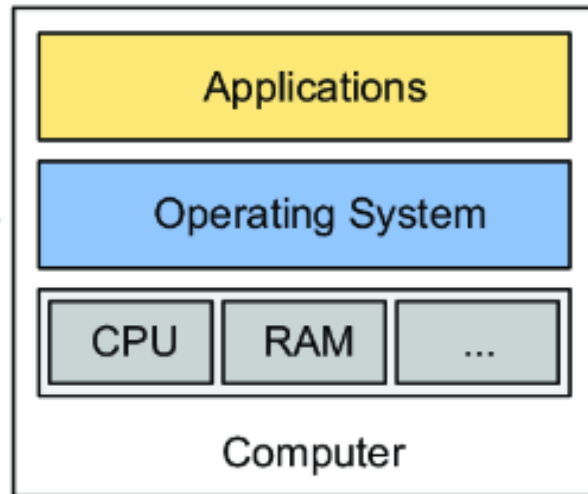
Understanding Kubernetes

Kubernetes is like an OS of a computer cluster

- Kubernetes is designed to run applications on a cluster of computers
- One can imagine Kubernetes as an operating system for the cluster

An operating system is an interface between applications and the underlying hardware.

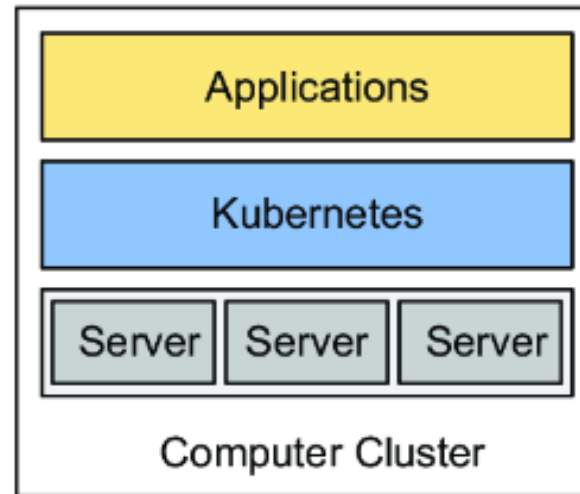
Hardware inside the computer



Computer

Kubernetes is an interface between distributed applications and the cluster of machines they run on.

Computers and other hardware that makes up a computer cluster



Computer Cluster

Figure 1.8 Kubernetes is to a computer cluster what an operating system is to a computer.

Main services

- Service discovery
 - A mechanism that allows application to find other applications and use the services they provide
- Horizontal scaling
 - Replicating your application to adjust to fluctuations in load
- Load balancing
 - Distributing load across all the application replicas
- Self healing
 - Keeping the system healthy by automatically restarting failed applications and moving them to healthy nodes after their nodes fail
- Leader election
 - A mechanism that decides which instance of the application should be active while the others remain idle but is ready to take over if the active instance fails

How Kubernetes fits in a cluster

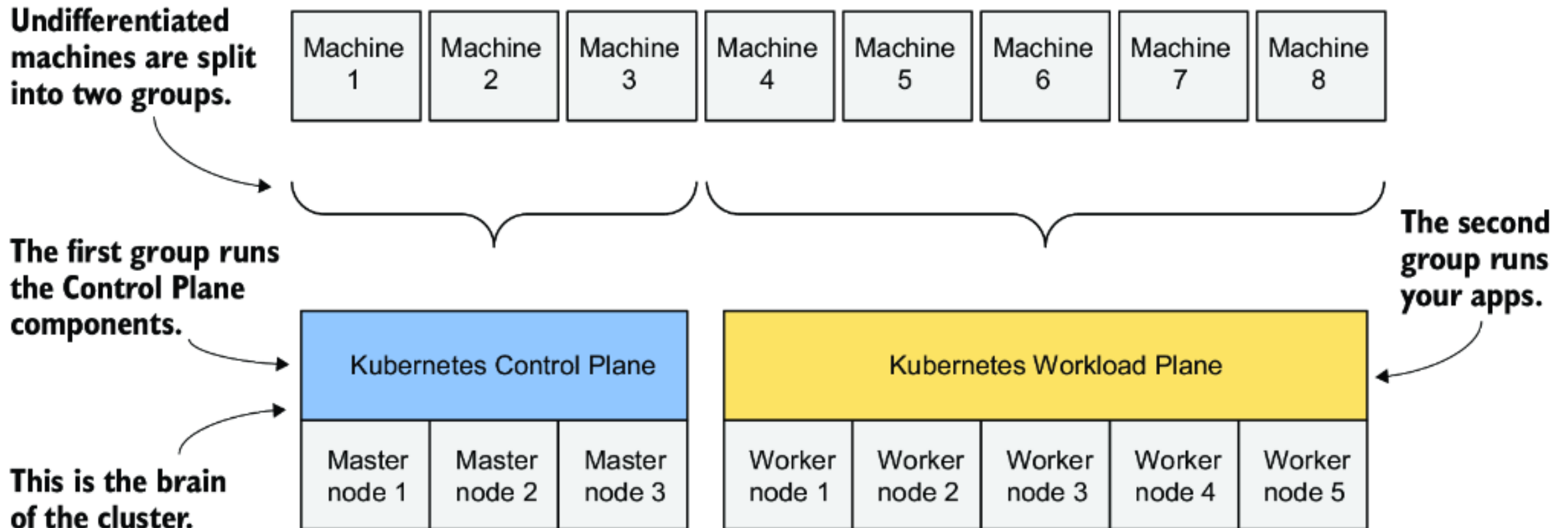
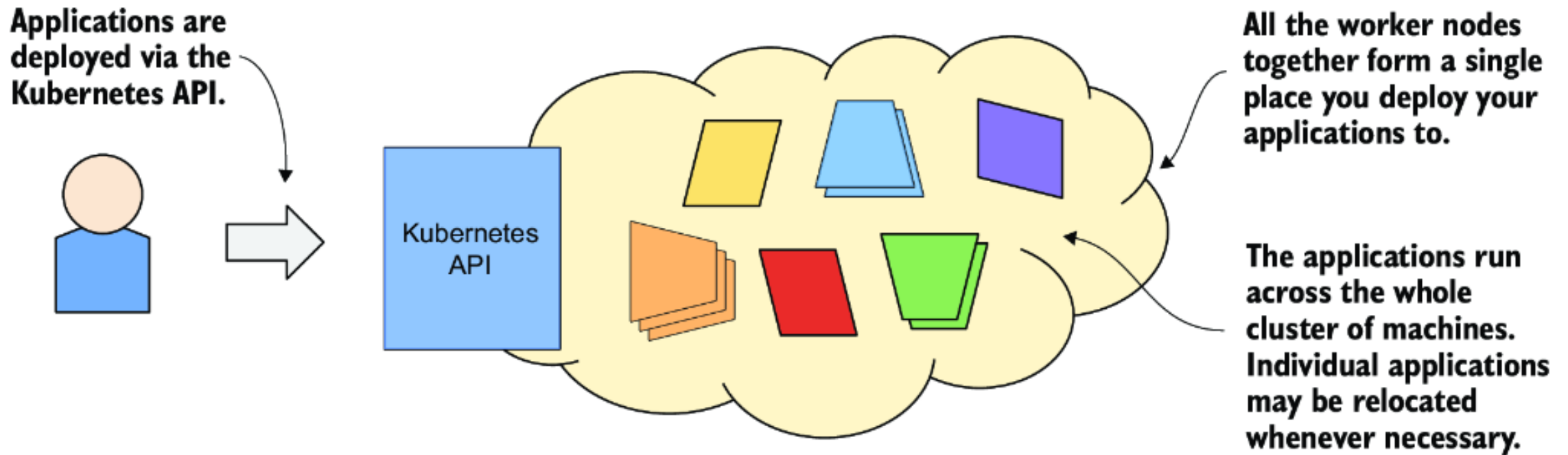


Figure 1.9 Computers in a Kubernetes cluster are divided into the control and the workload plane.

Simplest View of Kubernetes



Kubernetes expose the whole data center as a single deployment platform

Kubernetes Architecture

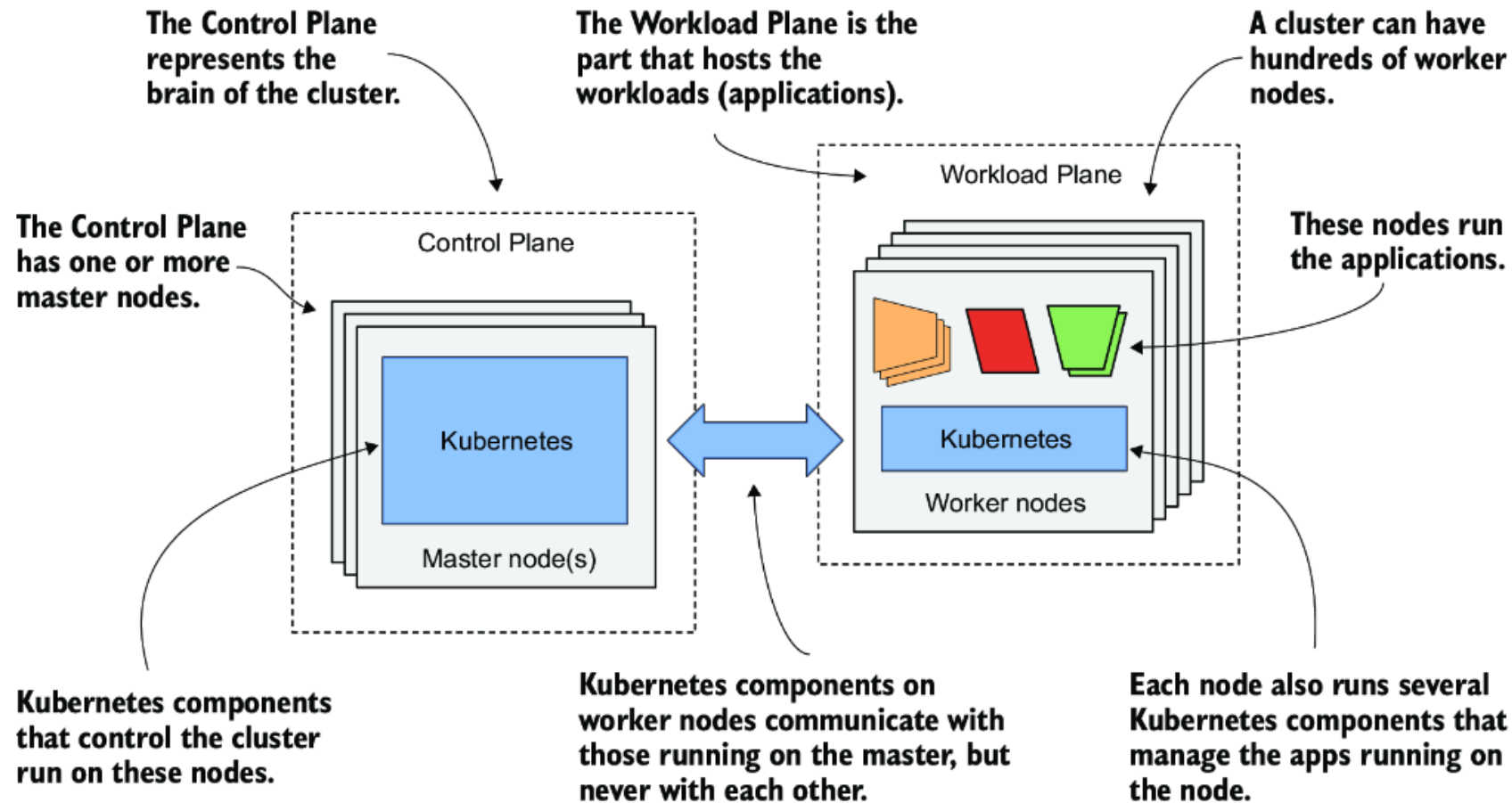


Figure 1.11 The two planes that make a Kubernetes cluster

Control plane components

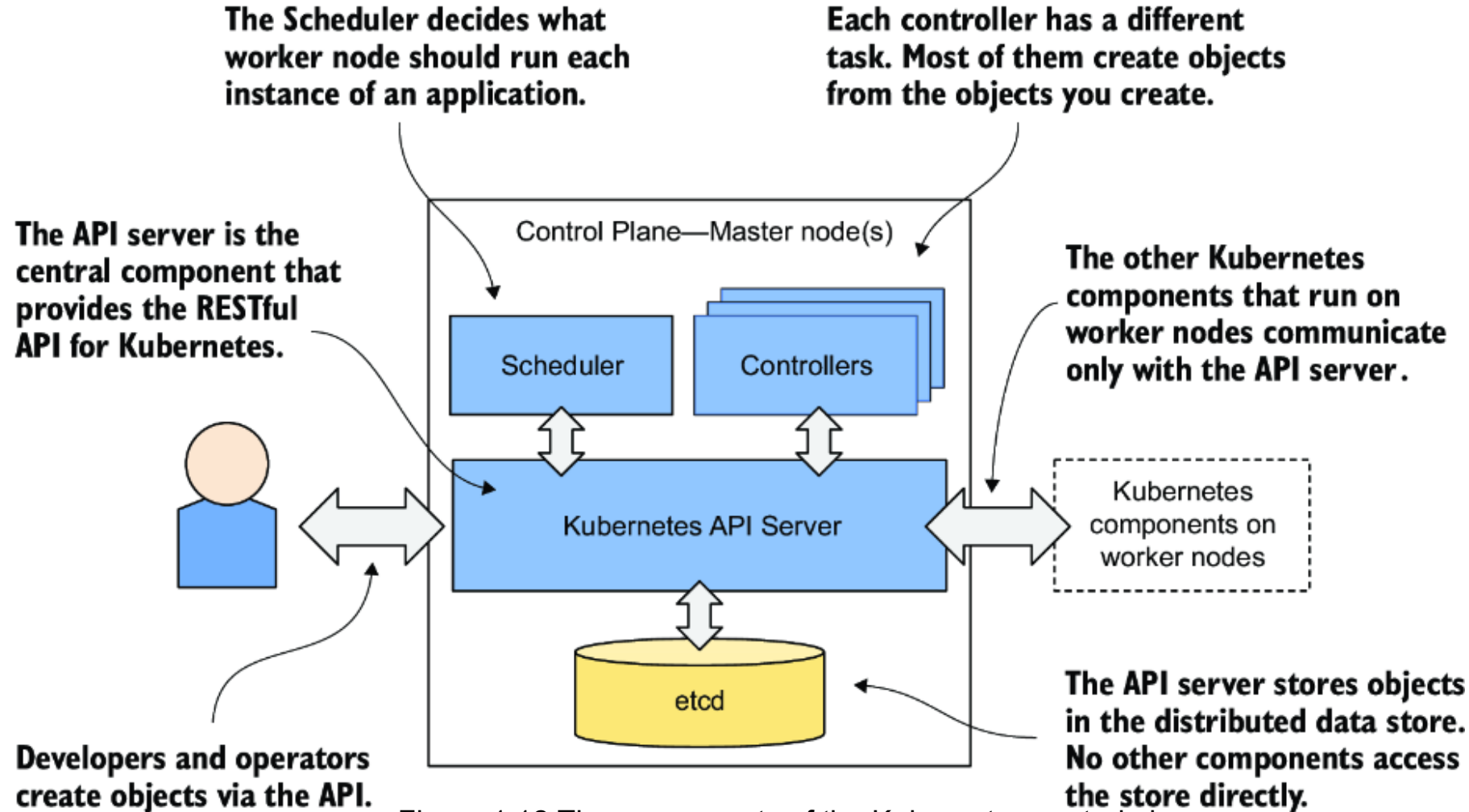
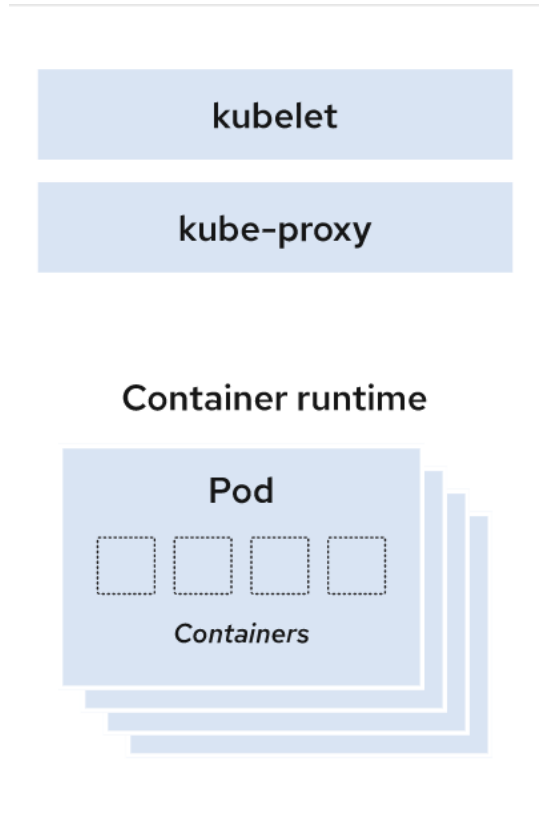


Figure 1.12 The components of the Kubernetes control plane

The Master Node (control plane)

- The *Kubernetes API Server*, which you and the other Control Plane components communicate with
- The *Scheduler*, which schedules the application to run on a set of worker nodes
- The *Controller Manager*, which performs cluster-level functions, such as replicating components, keeping track of worker nodes, handling node failures, and so on
- *etcd*, a reliable distributed data store that persistently stores the cluster configuration.

The Worker Node



- The worker node can be a virtual or a physical machine
- Each node is managed by the control plane and has a few system components to manage and run pod:
 - **Kubelet** talks to the API server and manages containers on its node. It utilizes *cgroups* and *tc (traffic control)* to reserve computing resources for the pod
 - **Kube-proxy** maintains network rules on nodes
 - **Container Runtime**: the software that is responsible for running containers, e.g. docker or others

Running an application in Kubernetes

- Basic steps
 - Package the application into one or more container images
 - Push those images to an image registry
 - Create Kubernetes objects to represents the application
 - Through simple command line command,
 - Or a manifest file

Running a simple application

```
kubectl create deployment kiada --image=luksa/kiada:0.1
```

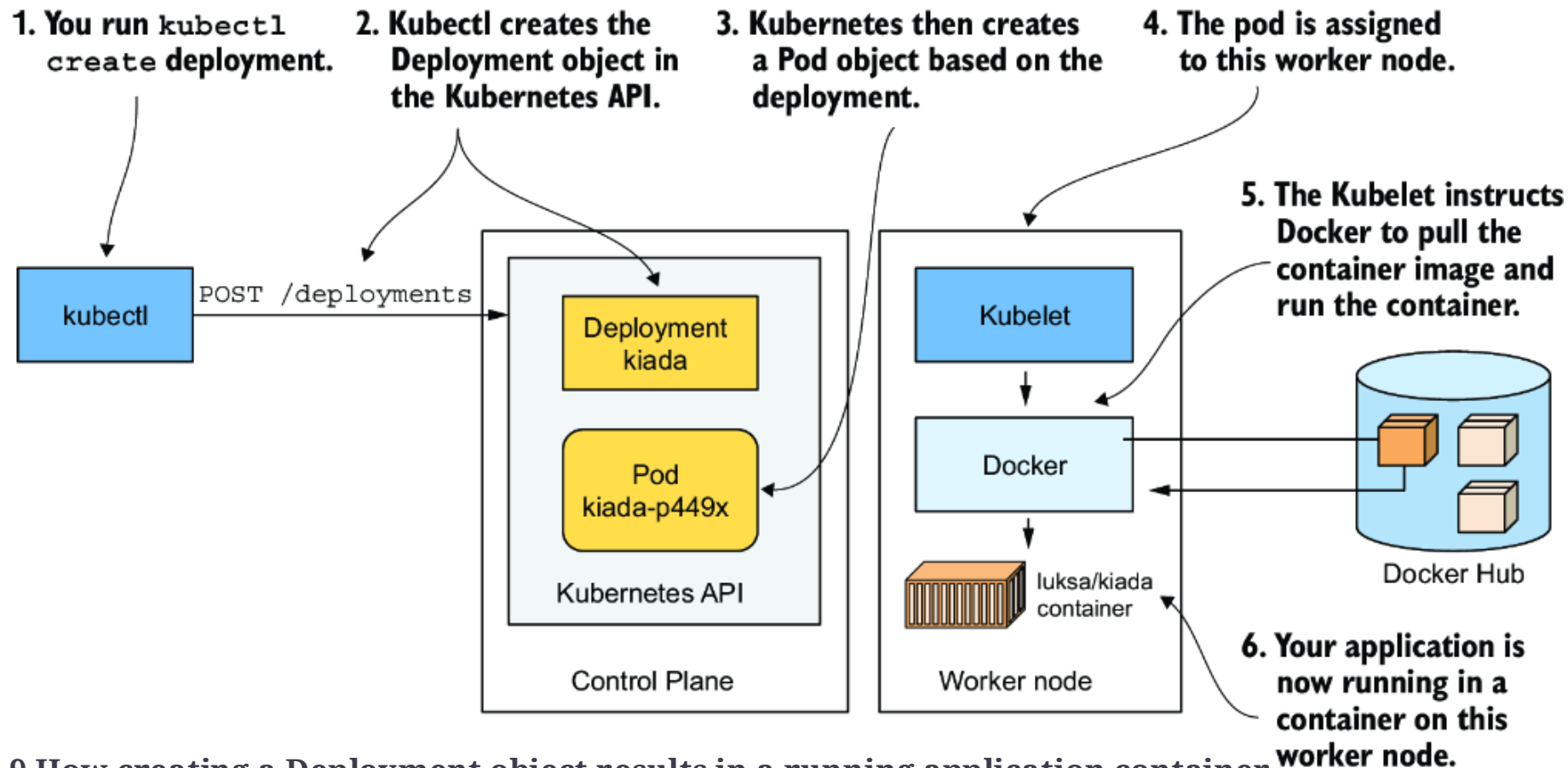


Figure 3.9 How creating a Deployment object results in a running application container

Kubernetes objects

- Kubernetes objects are entities created and managed by a Kubernetes cluster
- They usually represents things you need to run an application
 - Pods
 - Deployment
 - Service
 - Etc
- Some may be referred to as resources

A simple application on Kubernetes

- A simple application on Kubernetes are represented by three basic objects

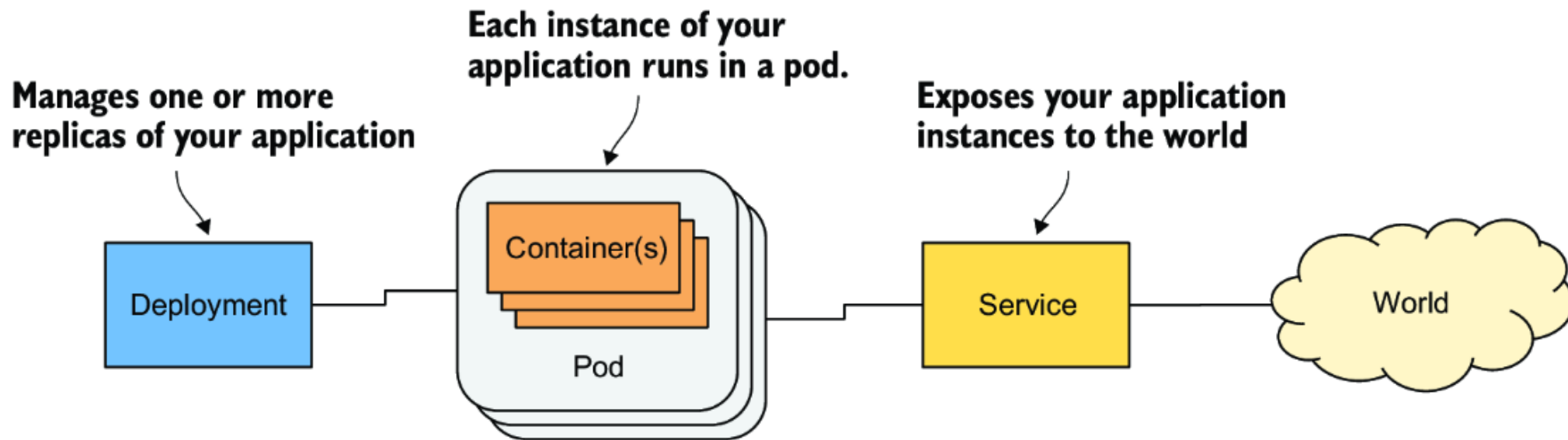


Figure 5.1 Three basic object types comprising a deployed application

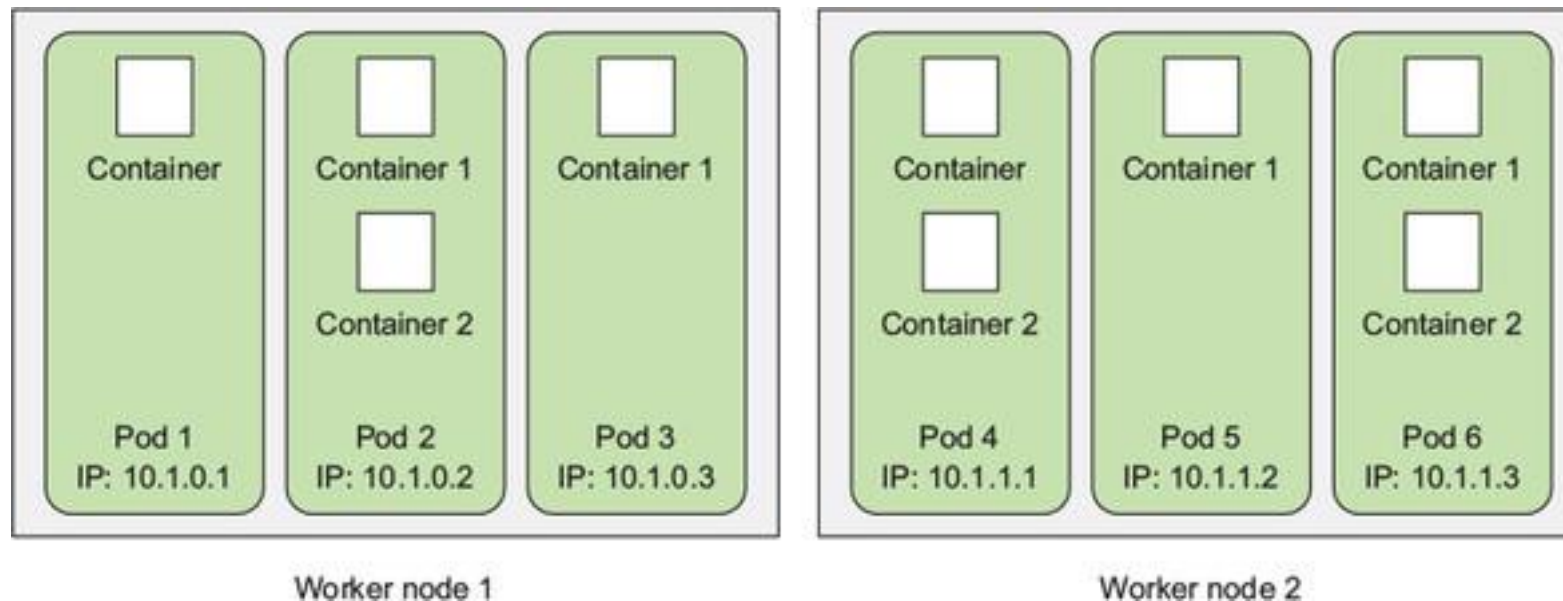
Pods

Pod and containers

- A **pod** is the unit of scheduling in Kubernetes. It is a resource envelope in which one or more containers run. Containers that are part of the same pod are guaranteed to be scheduled together onto the same machine, and can share state via local volumes.
 - E.g. web server that generates logs alongside a lightweight log collection process that ships the log to a cluster filesystem
- One node can run multiple pods

Node, Pod and Container

- A node is a virtual or a physical machine
- It can run many containers managed by a container run time
- Kubernetes use Pod to provide a logical grouping of containers that need to run on the same node



Understanding Pod

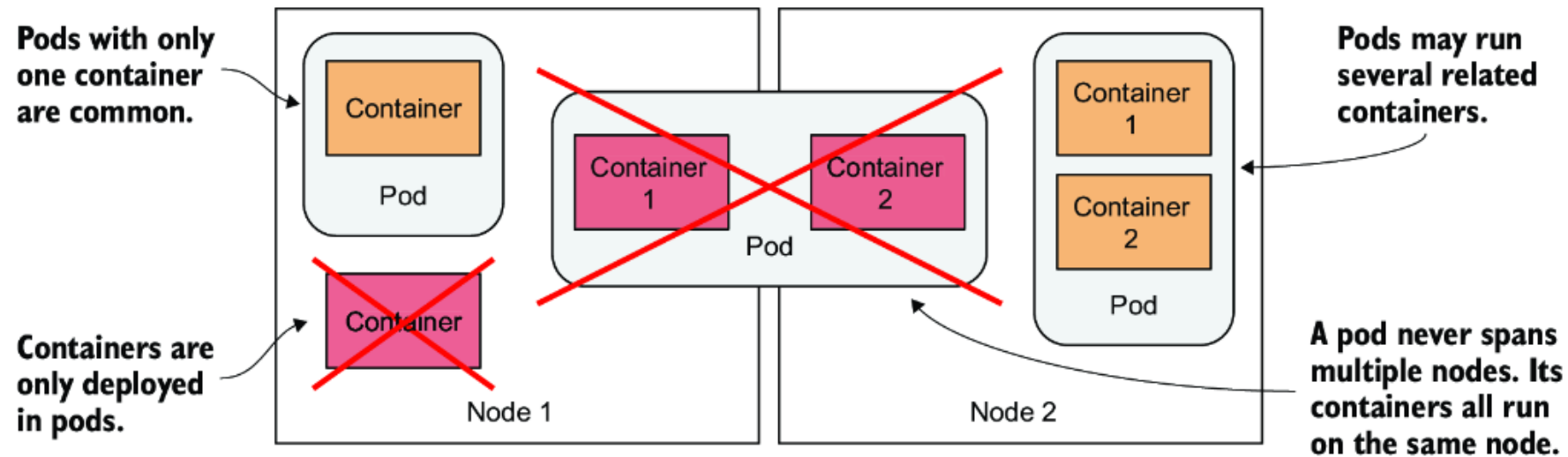


Figure 5.2 All containers of a pod run on the same node. A pod never spans multiple nodes.

Why we need pods? (1 of 2)

- Containers are designed to run only a single process per container
 - The process may spawn child processes
- Running multiple relatively independent processes in the same container defeats the purpose of containerization
 - The different processes may have different resource requirements
 - Certain mechanism is needed to monitor/restart individual process within the container
 - They can't be managed as a whole

Splitting an application into multiple pods

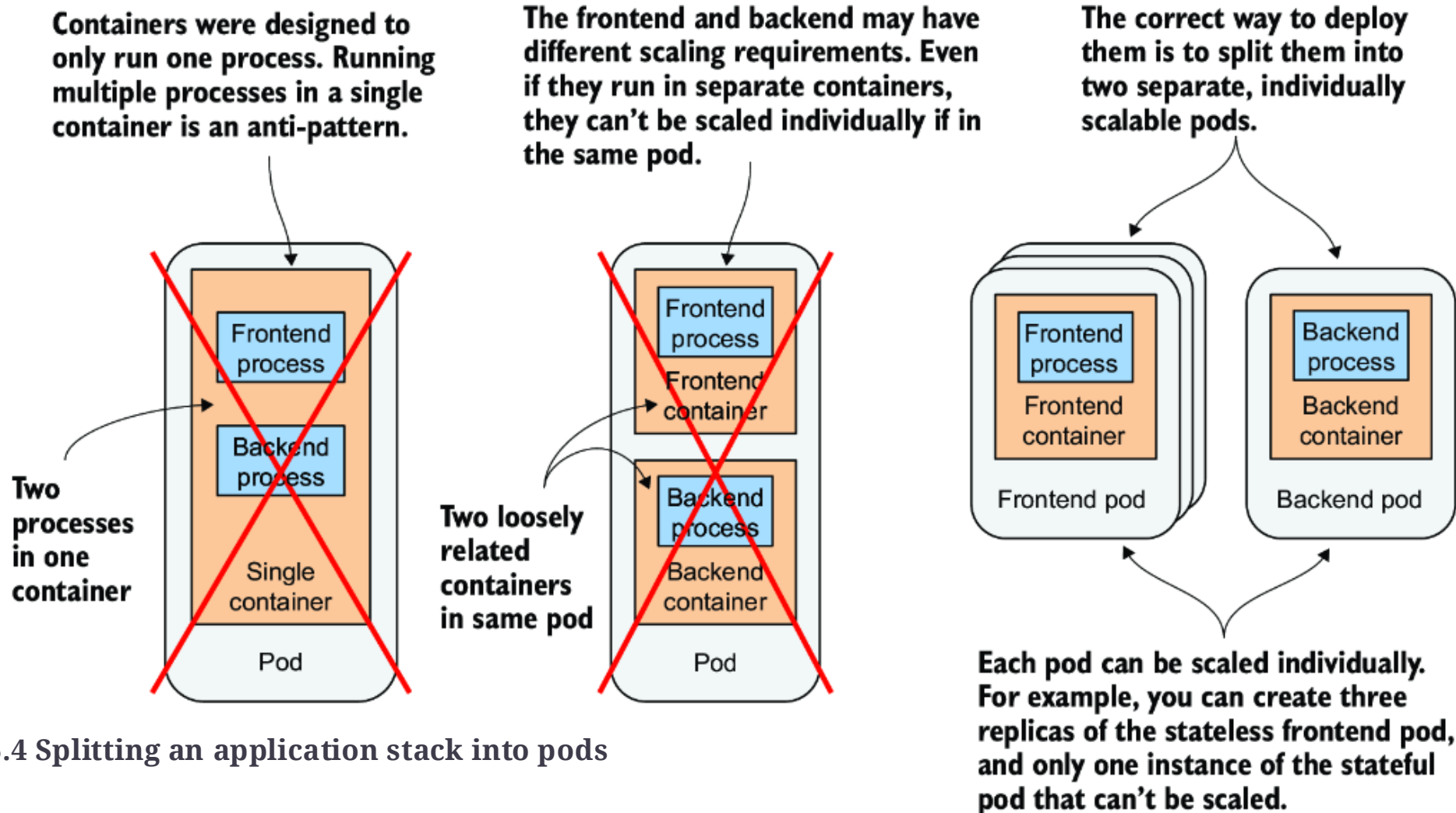


Figure 5.4 Splitting an application stack into pods

Why we need pods? (2 of 2)

- Processes need to run in their individual containers
- Some processes are more closely related than others; they may need to share some resources
- Pod is the higher- level construct to group them together
 - Take advantages of the container features
 - Giving processes the illusion that they are running together

Partial isolation within the same pod

- Containers are isolated from each other
 - Each gets its own copy of the OS resources using the namespace feature
 - Each has its own IP address
- Containers within the pod are partially isolated
 - They share the same namespace
 - They have a single IP address, which is the pod IP address

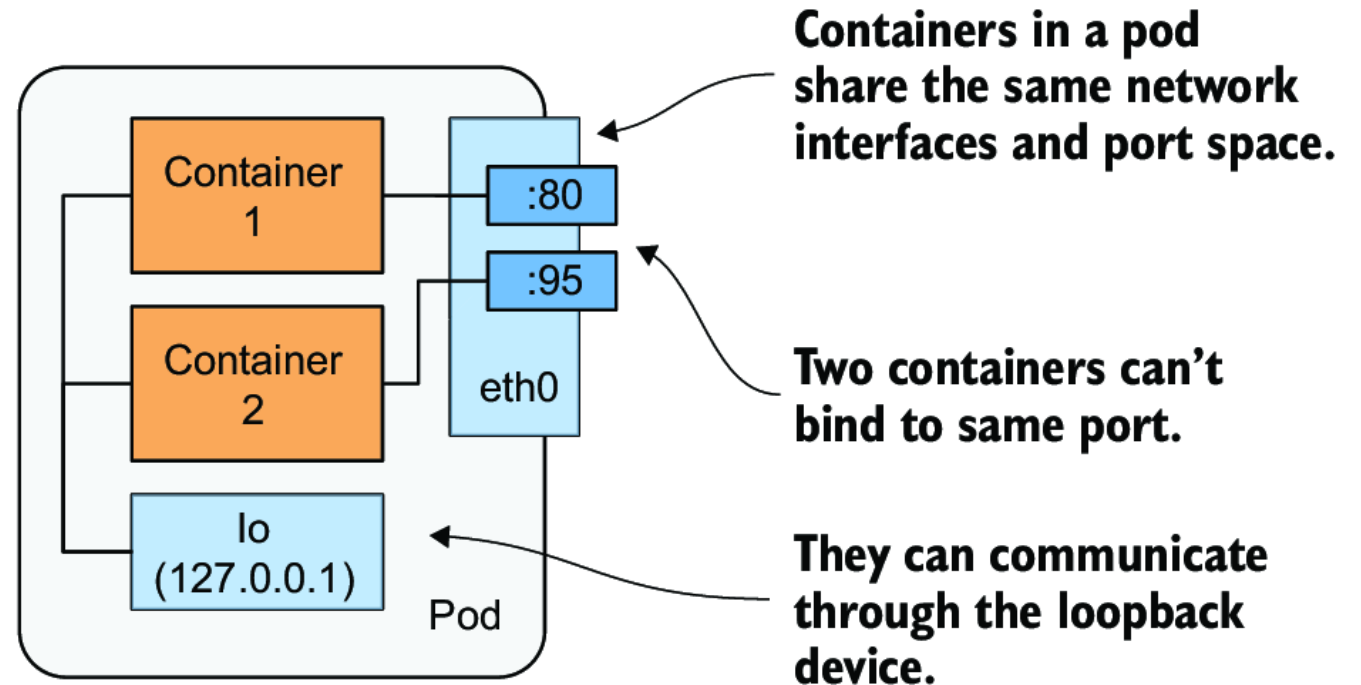


Figure 5.3 Containers in a pod share the same network interfaces.

Sidecar containers

- Placing several containers in a single pod is only appropriate if the application consists of a primary process and one or more processes that complement the operation of the primary process.
- The container in which the complementary process runs is called a *sidecar container*

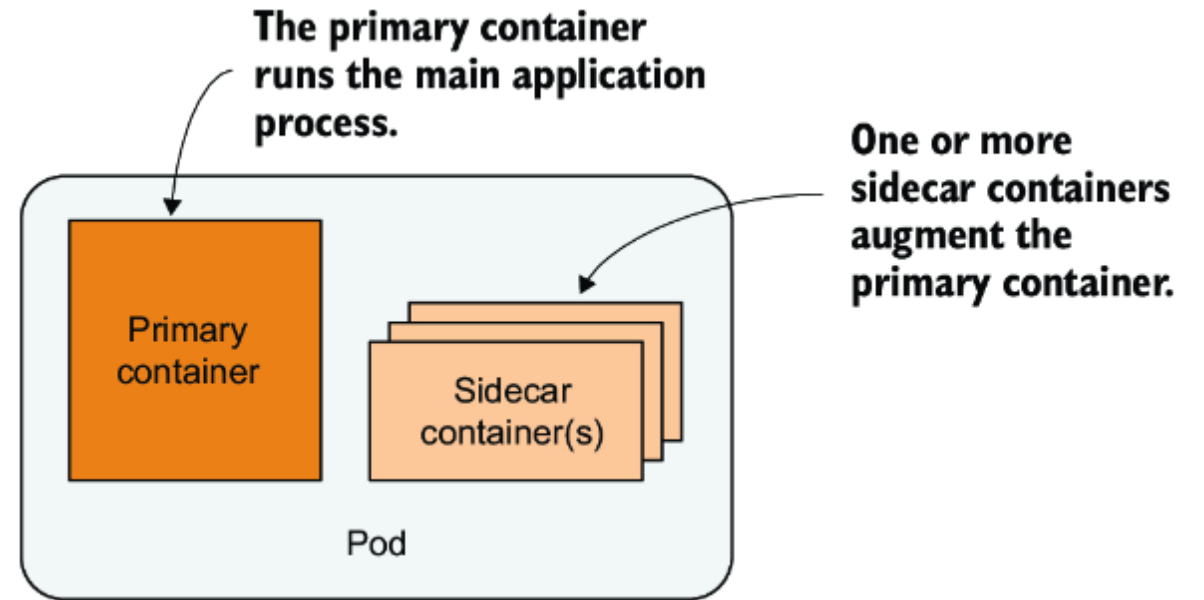


Figure 5.5 A pod with a primary and sidecar container(s)

Example sidecar container

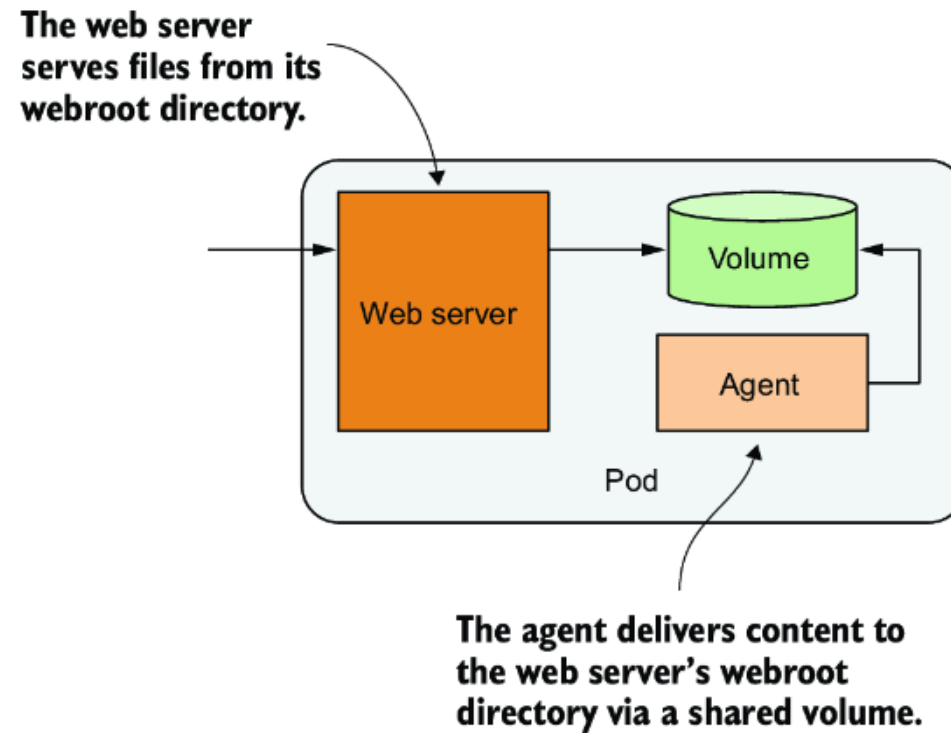


Figure 5.7 A sidecar container that delivers content to the web server container via a volume

Service and Deployment

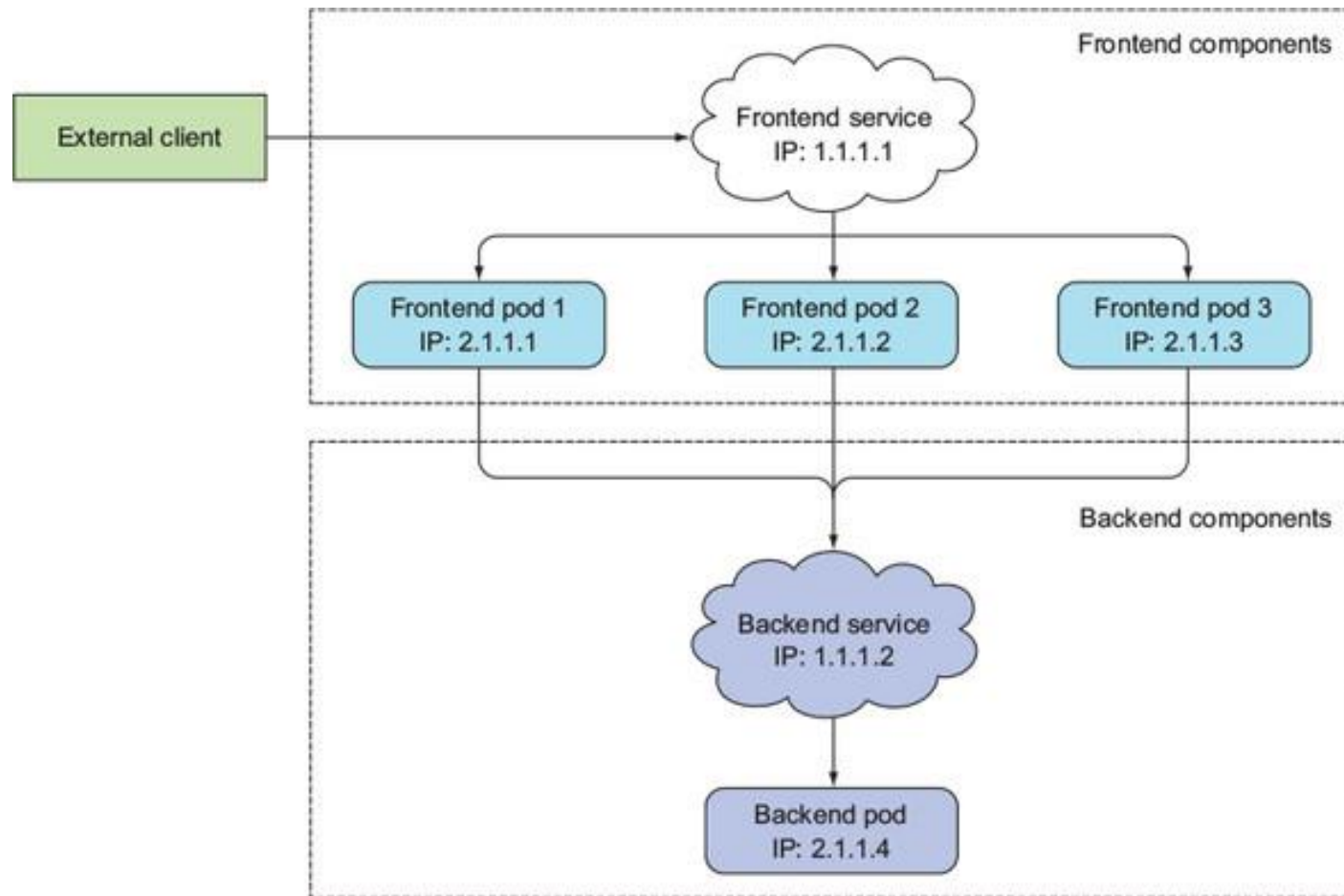
Deployment

- A deployment represents an application deployed in the cluster
- In microservices application, one Kubernetes deployment commonly represents one microservice
- An application/microservice may run on one or multiple pods
- A deployment provides a declarative way to specify the pod's resource requirements and desirable states
 - The container image
 - The node specification
 - The number of replicas

Service

- A deployment usually runs on one or multiple pods
 - Each pod receives its own IP address, which is a private one
 - Pods are ephemeral, they could be replaced by new pods on other nodes
 - One or more pods may represent the front-end service that needs a "permanent" external IP address
- Service is used to expose a group of pods (a deployment) as a single address
- A service is needed when
 - There are replicas for the pod and load balancing is needed
 - Some pods need to communicate with the external clients

Kubernetes service illustration



ECS vs Kubernetes

Concept	ECS	Kubernetes
Unit of deployment	Task	Pod
Long-running replicated component	ECS Service	Deployment
Container definition	Task definition	Pod template
Cluster machines	EC2 container instances or Fargate	Worker nodes
Load balancing	ALB/NLB target group	Service
Desired count	Service desired tasks	Deployment replicas

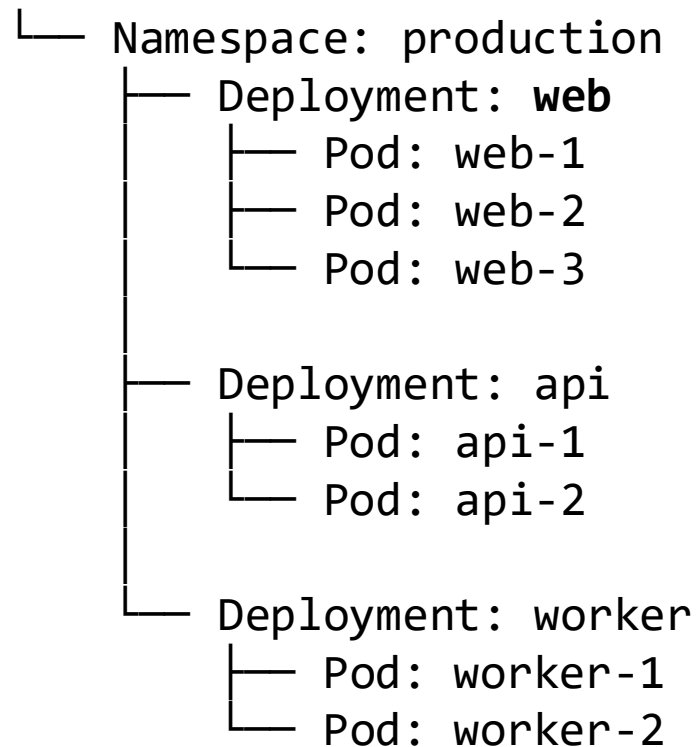
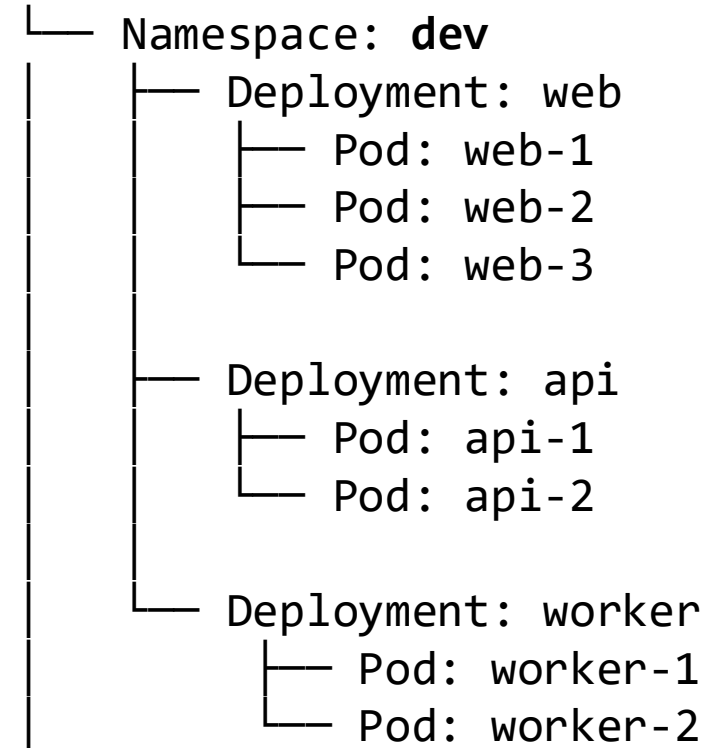
Kubernetes Manifest file

Features to group and associate objects

- A Kubernetes cluster manages objects belonging to different applications and users
- A number of features are provided to make it easier to manage them
 - Namespace
 - A mechanism for isolating groups of objects/resources within a single cluster
 - Used to avoid object name conflict
 - Label
 - An arbitrary key value pair that can be attached to objects
 - Usually hold identifying information
 - They are like AWS tags
 - Annotation
 - Also represented as key value pairs
 - Used for different purpose, e.g. providing the description of an object

Namespace, deployment and pod

Kubernetes Cluster



Object can be described in a manifest file

- YAML or JSON format
- The manifest should include the following fields:
 - apiVersion - Which version of the Kubernetes API you're using to create this object
 - kind - What kind of object you want to create
 - metadata - Data that helps uniquely identify the object, including a name string, UID, and optional namespace
 - spec - What state you desire for the object

<https://kubernetes.io/docs/concepts/overview/working-with-objects/#kubernetes-objects>

Example pod manifesto

```
apiVersion: v1      ❶
kind: Pod           ❷
metadata:
  name: kubia-manual  ❸
spec:
  containers:
  - image: luksa/kubia  ❹
    name: kubia         ❺
    ports:
    - containerPort: 8080  ❻
      protocol: TCP
```

- ❶ Descriptor conforms to version v1 of Kubernetes API
- ❷ You're describing a pod.
- ❸ The name of the pod
- ❹ Container image to create the container from
- ❺ Name of the container
- ❻ The port the app is listening on

Standalone pod manifesto is usually only used for short test

A pod with labels

```
apiVersion: v1
kind: Pod
metadata:
  name: kubia-manual-v2
  labels:
    creation_method: manual ❶
    env: prod ❶
spec:
  containers:
  - image: luksa/kubia
    name: kubia
    ports:
    - containerPort: 8080
      protocol: TCP
```

❶ Two labels are attached to the pod.

- creation_method=manual
- env=prod

Example deployment manifest

```
apiVersion: apps/v1beta1 ❶
kind: Deployment ❷
metadata:
  name: kuba ❸
spec:
  replicas: 3
  template:
    metadata:
      name: kuba
      labels:
        app: kuba
    spec:
      containers:
        - image: luksa/kuba:v1
          name: nodejs
```

- ❶ Deployments are in the apps API group, version v1beta1.
- ❷ the kind is Deployment.
- ❸ the name of the Deployment.

The template to create the pod

Create pod as part of a deployment or other files are more common

Note the version apps/beta1 is deprecated

The manifesto won't work in current Kuterbetes cluster

Another deployment manifesto

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

spec:

selector:

matchLabels:

app: nginx

Describe pods managed by the deployment

replicas: 2

Specifies the desirable number of pods for this deployment

template:

metadata:

labels:

app: nginx

Specifies the template used to create pods for this deployment

spec:

containers:

- name: nginx

image: nginx:1.14.2

ports:

- containerPort: 80

Example service manifesto

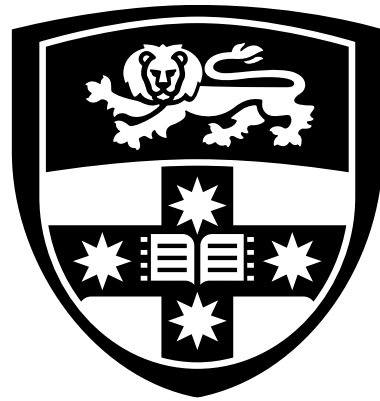
```
apiVersion: v1
kind: Service
metadata:
  name: kubia
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: kubia
```

The kind is Service

- ① The port this service will be available on
- ② The container port the service will forward to
- ③ All pods with the app=kubia label will be part of this service.

References

- Marko Luksa, Kubernetes in Action, first edition & second edition
 - Online accessed from Sydney University Library
 - Chapter 1, 2, 3



THE UNIVERSITY OF
SYDNEY